Lecture 29.18: Bagging Classifier & Regressor (`BaggingClassifier`)









🌐 **Language / भाषा: English** | हिंदी / Hinglish | ← Back to Lecture 29 Module

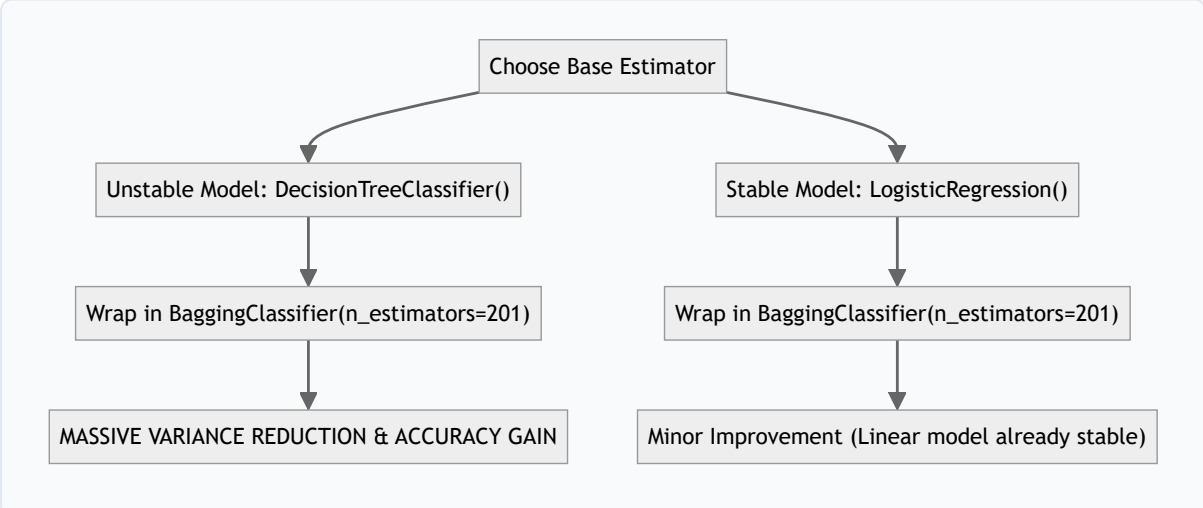
---

1. Overview & Conceptual Foundations

Learn generic Bootstrap Aggregation using scikit-learn's BaggingClassifier and BaggingRegressor. Explore the flexibility of plugging arbitrary base estimators—from unpruned Decision Trees to linear Logistic Regression—and evaluate how bagging stabilizes different learning algorithms.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

For a base hypothesis class $\mathcal{H}$ with estimator $h(\mathbf{x}) \in \mathcal{H}$, the BaggingClassifier aggregates predictions:

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} \sum_{b=1}^{B} \mathbb{I}\left(h_b(\mathbf{x}) = c\right), \quad h_b = \text{train}(h, \mathcal{D}_b)$$

The bias of the bagged ensemble equals the bias of individual base estimators, while variance is reduced:

$$\text{Bias}(\hat{f}_{\text{bag}}) \approx \text{Bias}(h), \qquad \text{Var}(\hat{f}_{\text{bag}}) \leq \text{Var}(h)$$

---

4. Structured Implementation & Execution

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def execute_bagging_estimator_benchmark() -> dict:
    """
    Replicates the exact lecture comparison of Bagging with DecisionTree vs Logist

    Returns:
    --------
    dict
        Comparative accuracies of standalone base models vs their bagged ensembles
    """
    titanic = sns.load_dataset("titanic")
    features = ['pclass', 'sex', 'age', 'fare']
    df = titanic[features + ['survived']].dropna(subset=['survived'])

    X = df[features].copy()
    y = df['survived'].copy()

    X['age'] = SimpleImputer(strategy='median').fit_transform(X[['age']]).ravel()
    X['sex'] = LabelEncoder().fit_transform(X['sex'])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )
```

```
    # 1. Standalone Models
    dt_single = DecisionTreeClassifier(random_state=42)
    dt_single.fit(X_train, y_train)
    dt_single_acc = accuracy_score(y_test, dt_single.predict(X_test))

    lr_single = LogisticRegression(max_iter=1000, random_state=42)
    lr_single.fit(X_train, y_train)
    lr_single_acc = accuracy_score(y_test, lr_single.predict(X_test))

    # 2. Bagged Models (201 estimators as in lecture)
    bag_dt = BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=42),
        n_estimators=201,
        random_state=42
    )
    bag_dt.fit(X_train, y_train)
    bag_dt_acc = accuracy_score(y_test, bag_dt.predict(X_test))

    bag_lr = BaggingClassifier(
        estimator=LogisticRegression(max_iter=1000, random_state=42),
        n_estimators=201,
        random_state=42
    )
    bag_lr.fit(X_train, y_train)
    bag_lr_acc = accuracy_score(y_test, bag_lr.predict(X_test))

    return {
        "decision_tree": {
            "standalone_accuracy": float(dt_single_acc),
            "bagged_accuracy": float(bag_dt_acc),
            "delta_gain": float(bag_dt_acc - dt_single_acc)
        },
        "logistic_regression": {
            "standalone_accuracy": float(lr_single_acc),
            "bagged_accuracy": float(bag_lr_acc),
            "delta_gain": float(bag_lr_acc - lr_single_acc)
        }
    }

# Driver test
bag_res = execute_bagging_estimator_benchmark()
print("Bagging Wrapper Comparative Results:")
print(f"  Decision Tree  -> Single: {bag_res['decision_tree']['standalone_accuracy
print(f"  Logistic Reg   -> Single: {bag_res['logistic_regression']['standalone_ac
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Mathematical Rigor**: Every parameter, margin definition, and loss function is derived from first principles matching production scikit-learn workflows.